

Technical and Editorial Review

Object-Oriented Design Patterns Seminar Report

August 29, 2026

Overall assessment

The manuscript is substantially developed and generally clear. The current `main.tex` compiles without LaTeX warnings to a 34-page PDF. The Facade and Strategy chapters now include general and example-specific TikZ diagrams, trade-off tables, application discussions, and seven-question quizzes. The Facade and naive programs compile under C++11 and produce the same device-output sequence. The explanation of `const RouteStrategy*` now correctly states that the pointer is mutable while access to the strategy through that pointer is const-qualified. The revised Facade wording also correctly distinguishes a new client-facing interface from new subsystem functionality.

The report is nevertheless not submission-ready. Its main technical example for Strategy demonstrates runtime dispatch but does not calculate routes, despite repeated claims that it does. The diagrams continue to omit ownership and constness information that is central to the accompanying discussion. The Decorator chapter remains incomplete against the explicit per-pattern requirements. The newly added AI-disclosure material also contains an incomplete prompt log presented as an “AI Chat Log,” and final-facing placeholders and artifact gaps remain.

No formal derivations, coordinate transformations, inverse-trigonometric expressions, or sign-sensitive equations occur in the manuscript. The required branch, quadrant, sign, symbol-definition, and dimensional checks were therefore either not applicable or limited to the numerical software examples. The audit explicitly checked all implementation signatures, ownership claims, diagrams, quantitative outputs, appendices, citations, and references. The required terminology-drift check identified the UML and ownership conflicts below.

Technical and evidence findings

1. The Strategy classes are dispatch stubs, not routing algorithms

Classification. Unsupported claim; major.

Location. `main.tex`, lines 755–774, 788–879, and 1053–1175; the four concrete strategy headers under `Seminar/Strategy/src/strategy/`.

Problem. Every concrete `buildRoute(origin, destination)` ignores both parameters, returns `void`, and prints a fixed description, distance, and travel time. For example, `DrivingStrategy` prints “25 mins (18.5 km)” for every possible origin and destination. A C++11 build with `-Wall -Wextra`

-Wpedantic produces eight unused-parameter warnings. The cited Refactoring.Guru Strategy page describes its navigation example as returning route checkpoints from the supplied origin and destination, whereas the seminar implementation produces no route result at all.

Why it matters. The code establishes interchangeable runtime behavior, but it does not support claims about route calculation, traffic handling, timetable evaluation, elevation optimization, estimates, or testing of actual routing algorithms. The fixed numbers appear empirical even though they are unrelated to input or data.

Suggested fix. Either call these classes pedagogical Strategy stubs and remove the fabricated route estimates, or implement an input-dependent contract. A minimal credible interface would return a `RouteResult` containing a path or checkpoints, distance, duration, and status. Each concrete strategy should consume the locations or an injected data source. Restrict the performance and testing discussion to behavior that the implementation actually provides.

2. The UML diagrams still contradict material code semantics

Classification. Definite internal inconsistency; major.

Location. Decorator diagram, lines 223–241; home-theater diagram, lines 479–573; Strategy diagrams and explanation, lines 882–1051.

Problem.

- The Decorator diagram gives `wrapped` the type `Beverage*`, while the implementation uses `std::unique_ptr<Beverage>` and explicitly claims exclusive ownership.
- The home-theater diagram shows value-like members such as `Amplifier amp`; the implementation stores five references, including `Amplifier& amp`, and relies on non-owning lifetime semantics.
- The navigation diagram shows `RouteStrategy*` and `setStrategy(RouteStrategy*)`; the code uses `const RouteStrategy*` in both places.
- The actual client constructs all four concrete strategies, but the navigation diagram draws client dependencies only to `DrivingStrategy` and `BicyclingStrategy`.
- The prose says the hollow diamond “expresses” delegation. A hollow diamond denotes shared aggregation; delegation is the behavioral call from `Navigator` to `RouteStrategy`. The diagram and prose should not conflate those concepts.

Why it matters. Ownership, lifetime, mutability, and delegation are explicit teaching objectives. The diagrams currently teach different types and relationships from the code, so a reader cannot reliably implement the design from them.

Suggested fix. Preserve the exact pointer/reference and `const` qualifiers in implementation diagrams. Show `unique_ptr` ownership for Decorator, references or a clearly annotated non-owning association for Facade, and `const RouteStrategy*` for Strategy. Either draw all concrete-strategy client dependencies or replace them with one documented generic creation dependency. Use a plain directed association for the non-owning Strategy link unless shared aggregation is intentionally taught, and describe delegation separately in the text.

3. The Decorator listing has an unstated C++14 requirement

Classification. Definite compatibility issue if the seminar standard is C++11; otherwise implementation ambiguity.

Location. `main.tex`, lines 249–283.

Problem. The listing uses `std::make_unique`, which is available from C++14, while the other implementations are explicitly built as C++11. Rechecking the displayed Decorator code with `g++-std=c++11` fails at both `std::make_unique` calls. The excerpt also omits `<memory>`, `<string>`, and `<utility>` if it is intended to be independently compilable.

Why it matters. Readers are given no indication that one of the three examples requires a different language standard, and the assignment requests source code rather than an untested fragment.

Suggested fix. State and use C++14 consistently, or rewrite construction for C++11. Add the necessary includes, a complete `main`, expected output, and a reproducible compile command. Prefer a standalone Decorator source directory parallel to the Facade and Strategy examples.

4. Reference members do not guarantee dependency validity

Classification. Likely misleading claim.

Location. Facade implementation discussion, lines 618–621.

Problem. The report says references “guarantee non-null dependencies.” References make an explicit null state unavailable during ordinary well-defined construction, but they do not enforce the lifetime of the referred objects. Invalid references can also arise through prior undefined behavior. The following sentence correctly acknowledges that every subsystem must outlive the Facade.

Why it matters. The current wording sounds like a runtime safety guarantee, even though the central remaining risk is precisely a dangling non-owning reference.

Suggested fix. Write: “References make absence unrepresentable in ordinary well-defined construction and communicate non-ownership; the subsystem objects must outlive the Facade.” Keep null-state and lifetime guarantees separate.

5. The demonstration headers rely on transitive includes

Classification. Likely portability issue.

Location. `Seminar/Facade/src/facade/Projector.h`, line 10, its naive counterpart, and `Seminar/Facade/src/facade/HomeTheaterFacade.h`, line 22.

Problem. These headers name `std::string` without directly including `<string>`. They compile under the current include order because other headers happen to provide the declaration.

Why it matters. A header should be self-contained. Otherwise an unrelated include-order or standard-library implementation change can break a previously successful build.

Suggested fix. Add `#include <string>` to each header that names `std::string` directly, then syntax-check every project header when included first in a minimal translation unit.

6. Quantitative values are not consistently defined or derived

Classification. Implementation ambiguity; minor but concrete.

Location. Decorator prices at lines 258–282; Strategy outputs at lines 1071–1091 and in the source headers.

Problem. Decorator values 2.00 and 0.40 have no stated currency or unit. Strategy times and distances have units, but they are fixed literals rather than calculations. The Facade light level is consistently described as a percentage and presents no dimensional conflict.

Why it matters. An undefined price unit prevents interpretation, while fixed route estimates may be mistaken for computed or measured results.

Suggested fix. Define the beverage currency and mark all values as sample data. If the price example is intended beyond demonstration, use integer minor units or a decimal money representation. Remove route estimates until they are derived from inputs, or label them explicitly as canned output.

Coverage and submission findings

7. Decorator still fails several explicit per-pattern requirements

Classification. Definite completeness problem; major.

Location. Seminar/Requirements.md, items 1–10 under “For each pattern”; the complete Decorator chapter, `main.tex` lines 199–294.

Problem. Decorator has no 5–8 question multiple-choice quiz, no separate general and beverage-specific class diagrams, and no developed explanation of web and mobile applications. Its naive designs are prose-only despite the stated objective of comparing naive and pattern-based C++ designs. No standalone compilable Decorator source accompanies the source directories for the other patterns.

Why it matters. These elements are required for each pattern. The new Facade and Strategy material makes the short Decorator chapter’s incompleteness more visible and creates a section-to-section imbalance.

Suggested fix. Bring Decorator to the same evidence level: add a naive C++ implementation, separate general and problem diagrams, tested pattern source, explicit advantages and disadvantages, explained web/mobile examples, and a 5–8 item ABCD quiz with an answer key.

8. The embedded prompt list is not a complete AI Chat Log

Classification. Definite internal inconsistency; possible submission compliance issue.

Location. AI Usage Notes, lines 1339–1380; AI Chat Log, lines 1391–1515.

Problem. The chapter contains user prompts only, not the assistant responses or resulting actions, so it is a prompt inventory rather than a chat log. One item says “Timestamp: not exposed.” The declared access period ends at 09:33 UTC on August 29, but the latest displayed prompt is timestamped 04:18 UTC; the statement that available individual access times are recorded in the

following log therefore does not reconcile with the entries. The disclosure itself says that the complete Prism conversation still needs to be exported or captured separately.

Why it matters. The requirements explicitly request an AI Chat Log. A partial prompt list can be mistaken for the required complete record and cannot show what content the AI produced or what the group accepted.

Suggested fix. Rename the appendix “Prompt Summary” unless it is expanded to include the complete conversation. Reconcile the access-period end with the final logged event, explain any unavailable timestamp, and submit the actual exported conversation as the authoritative log. Confirm the required format with the instructor before submission.

9. Final-facing placeholders and artifact claims remain unresolved

Classification. Definite completeness and consistency problem.

Location. Group-member tables, lines 107–140; Submission Checklist, lines 1317–1330; Member Contribution Record, lines 1517–1522.

Problem. Both responsibility fields still contain bracketed example instructions. The contribution table already gives names and 50% shares, but the later appendix still says names and percentages “should be entered.” The checklist states that remaining artifacts are maintained separately, yet `Changes.md`, explicitly required by the assignment, is absent from the workspace at review time.

Why it matters. These contradictions are visible evidence of an unfinished submission and weaken the reliability of the checklist.

Suggested fix. Replace the responsibility placeholders with verified contributions, update or remove the stale contribution instruction, create and review `Changes.md`, and convert the checklist into an evidence-based status list rather than a general assertion.

10. Citation provenance does not match the revised content

Classification. Definite citation-integrity problem, with some claims requiring external verification.

Location. Pattern definitions and application sections; AI Chat Log, especially the Facade comparison prompt at lines 1490–1514; references, lines 1524–1547.

Problem. The body uses only `gof` and `refactoringguru`. The bibliography entries `headfirst`, `source-making`, and `cpp` are unused. The `refactoringguru` entry is specifically the Strategy page, although the recorded work also used the separate Facade page to add the Additional Facade participant and revise Facade terminology; that material has no matching body citation or bibliography entry. The Decorator intent and broad application list are also uncited.

Why it matters. A bibliography entry supports a claim only when the text cites the correct source. The current list does not let readers distinguish formal definitions, implementation facts, and illustrative analogies.

Suggested fix. Add and cite a separate Facade source, cite the source of the Decorator intent, and cite version-specific C++ claims if the ISO entry is retained. Support or qualify application examples such as Python decorators, API Gateways, ORMs, and mobile APIs. Remove genuinely unused entries and add access dates if required by the course style.

Meaningful editorial issues

AI-log naming. The heading “AI Chat Log” overstates a section that contains only selected prompts. “Prompt Summary” is accurate unless the full two-sided transcript is inserted. This change affects meaning and compliance, not merely style.

Algorithm terminology. Until route computation exists, replace “routing algorithm” and “calculates routes” near the demonstration with “strategy stub,” “mode-specific demonstration behavior,” or similarly bounded wording.

Participant naming. Use role names such as “Concrete Component” in prose and class-like identifiers such as `ConcreteComponent` only when showing UML or code. Consistent typography would make the general roles easier to distinguish from the beverage classes.

Proofing sweep

The bundled mechanical scan was run against the compiled 34-page manuscript. Its 80 double-period hits were table-of-contents leader dots, not punctuation defects. Source-level spot checks found no genuine duplicated comma, semicolon-capitalization anomaly, malformed “into . . . into” frame, JavaScript/iOS/iPhone capitalization problem, or `arctan(x/y)` branch ambiguity. The bibliography contains no obvious duplicated punctuation or broken quotation punctuation. The following genuine, high-confidence defects remain:

- **Location:** `Seminar/Strategy/src/strategy/BicyclingStrategy.h`, line 10. **Problem:** “elevation elevation profile optimization” repeats a word. **Fix:** use “elevation-profile optimization” or “elevation profile optimization.”
- **Location:** member tables, lines 120–133. **Problem:** bracketed “[Example: . . .]” instructions remain in submission-facing content. **Fix:** replace them with final verified responsibility statements.
- **Location:** Member Contribution Record, line 1521. **Problem:** “Member names and percentages should be entered” is stale because the earlier tables already contain names and 50% values. **Fix:** replace it with the final contribution summary or a reference to the completed official form.

Highest-priority fixes

1. Make the Strategy claims honest relative to the code: implement input-dependent route results or explicitly present the classes as dispatch stubs.
2. Complete Decorator so it satisfies every per-pattern assignment item and provide reproducibly compilable source.
3. Align all UML types and relationship semantics with ownership, lifetime, and constness in the implementations.
4. Replace the prompt-only appendix with, or clearly distinguish it from, the required complete AI Chat Log.

5. Resolve responsibility placeholders, the stale contribution statement, and the missing `Changes.md` artifact.
6. Repair citation provenance and the remaining C++ header/standard issues.

Items requiring external verification

Confirm with the instructor whether the AI Chat Log must contain the full two-sided transcript, screenshots, an export file, or only prompts, and whether timestamps and exact model/platform labels are mandatory. Verify the required reference style, URL policy, and access-date format. Finally, verify broad real-world pattern examples—especially Python function decorators, API Gateways, ORMs, camera APIs, and middleware—against authoritative documentation or design-pattern literature. They may be useful analogies, but the manuscript currently does not establish that each is a canonical Gang of Four instance.